MIDTERM – 2020 (SOLUTIONS)

Question 1 (**parts a and b are unrelated**).

a. Assume that multiply instructions take 20 cycles and account for 15% of the instructions in a typical program and that the other 85% of the instructions require an average of 4 cycles for each instruction. Your hardware engineering team has indicated that it would be possible to reduce the number of cycles required for multiplication to 8, but this will require 45% increase in the cycle time; and nothing else will be affected. Should they proceed the modification? Why or why not?

b. Assume the following instruction mix of a given program.

ALU	Load/Store	Branch	Jump
20%	30%	40%	10%

Assume the CPI values of the instruction types on two machines are given below:

Machine	Clock Speed	ALU	Load/Store	Branch	Jump
A	3 Ghz	3	X	3	2
B	1.2 Ghz	1	1	2	2

What should be the CPI of Load/store for machine A (i.e. the value of X) so that A and B have the same performance for the given program. *Show all your work to get full credit.*

a) $\#\text{of clock cycles}_{old} = 0.15\,I \times 20 + 0.85\,I \times 4$ (I → # of instructions.)

$= 3I + 3.4I = 6.4\,I$

$\#\text{of clock cycles}_{new} = 0.15I \times 8 + 0.85I \times 4$

$= 1.2I + 3.4I = 4.6\,I$

$CPU\ time_{old} = 6.4\,I \times CCT$

$CPU\ time_{new} = 4.6\,I \times (1.45 \times CCT) = 6.67\,I \times CCT$

$CPU\ time_{new} > CPU\ time_{old}$

$\Rightarrow$ Do not choose the modification

---

b) $\#\text{of clock cycles}_{A} = 3 \times 0.2 + X \times 0.3 + 3 \times 0.4 + 2 \times 0.1$

$= 2 + 0.3X$

$\#\text{of clock cycles}_{B} = 1 \times 0.2 + 1 \times 0.3 + 2 \times 0.4 + 2 \times 0.1$

$= 1.5$

A & B have the same performance

$$\frac{2 + 0.3X}{3} = \frac{1.5}{1.2} \qquad X \approx 5.8$$

Question 2 (**parts a,b and c are unrelated**).

a. In a 32-bit IEEE 754 floating-point system, what number does the bit pattern 0x3ea00000 represent? Show how you obtained your answer.

b. Consider the segment of assembly code below. To the left of each instruction we write the memory address where each instruction is stored. Write the binary representation of bne instruction (with all fields).

Memory Address	Label	Instruction
0000 0100	back:	add $s2, $s3, $s4
.....		
0011 0000		bne $s1, $zero, back

a) 0x3ea00000

[0011 1110 1010 0000 0000 0000 0000 0000]

0 | 01111101 | 010 0000 0000 0000 0000 0000

sign → 125

$= (-1)^0 \times (1 + 0.01) \times 2^{125-BIAS} = (1+0.01) \times 2^{-2}$

$= 1.01 \times 2^{-2} = 0.0101 = 2^{-2} + 2^{-4} = 0.25 + 0.0625 = 0.3125$

0.010 → 127

2^{-1} 2^{-2} 2^{-3} 2^{-4}

---

b) 0000 0100 back: → 4

0011 0000 → 48

(PC+4) → 48+4 = 52

(52−4)/4 = 12 ← addr in words

Thefe −12 ⇒ [1111 1111 1111 0100] = −12 (in 16 bits)

bne

opcode = 5

000101	10001	00000	1111 1111 1111 0100

↑ $s1 ↑ $zero

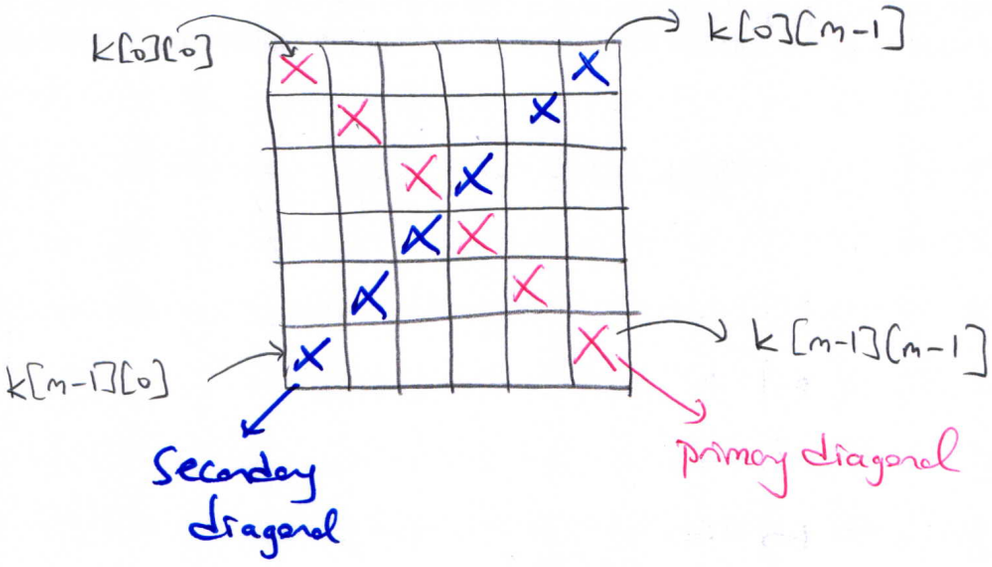
Question 3

Given that K is a square matrix of $M \times M$ integers (M rows by M columns), which is already read and initialized in memory. The starting address of matrix K is stored in register `$s2`, and M is stored in register `$s3`.

Write MIPS code to compute the sum of all elements on both primary and secondary diagonals of the matrix K. (The sum should be computed in `$s0`).

Add comments next to each MIPS code line briefly. Do not use pseudo-instructions.

Hint: Primary diagonal is the one that goes from K[0][0] to K[M – 1][M – 1] and the secondary diagonal goes from K[0][M – 1] to K[M – 1][0].



Store addr of
K → $s2
M → $s3

```
        add  $s0, $zero, $zero
        addi $t1, $s3, 1      # t1 = m+1
        addi $t2, $s3, -1     # t2 = m-1
        sll  $t1, $t1, 2      # (m+1) * 4
        sll  $t2, $t2 2       # (m-1) * 4
        add  $t3, $t2, $s2    # addr of k[0][m-1]
        add  $t4, $s2, $zero  # addr of k[0][0]
        add  $s1, $s3, $zero  # s1 = m
Sum:    lw   $t5, 0($t3)      # (now: k[0][m-1])
        add  $s0, $s0, $t5    # s0 = s0 + k[0][m-1]
        lw   $t5, 0($t4)      # k[0][0]
        add  $s0, $s0, $t5    # s0 = s0 + k[0][0]
```

↓ CONTINUE

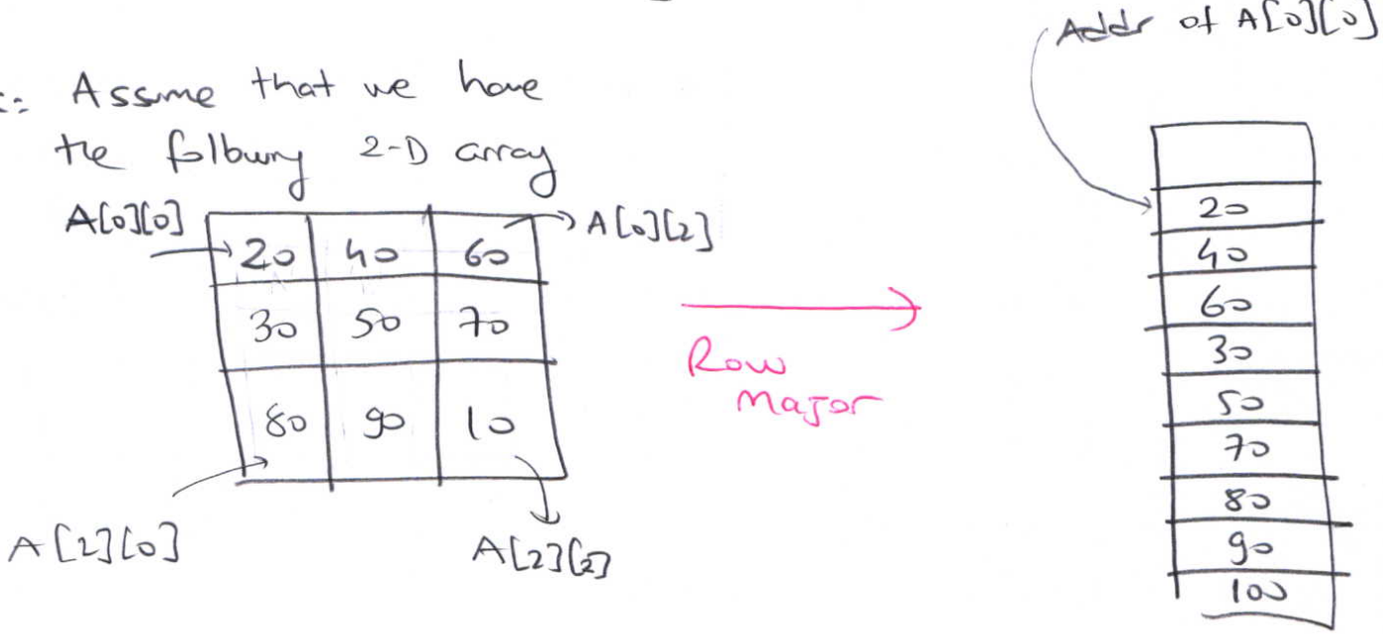
Explanation (Note: This part helps you to understand the solution. It is not part of the solution)

Note: the elements of a multi-dimensional array are stored in consecutive memory locations in memory

Row major order = assign array elements to memory locations row-by-row

Column major order = assign array elements to memory locations column-by-column

Ex: Assume that we have the following 2-D array



---

* we consider row-major order *

⇒ CONTINUE

```
add $t3, $t3, $t2
```

→ Consider secondary diagonal k[0][n-1] then k[1][n-2]

addr difference? = (n-1) × 4

```
add $t4, $t4, $t1
```

→ Consider primary diagonal k[0][0] ... k[1][1]

addr difference = (n+1) × 4

```
addi $s1, $s1, -1
bne $s1, $zero, Sum
```

→ for next iteration

Question 4 (a,b are unrelated)

a. Assume that the variables f, g, i, and j are assigned to registers $t0, $t1, $t2, $t3, and [redacted], respectively. Assume that the base address of the arrays A and B are in registers $s0 and $s1, respectively. Translate the following C code to MIPS code.

f = g + A[B[i]-B[j]];

b. The control flow in a MIPS program can be changed with several different instructions. Which instruction among (beq, bne, jal, jr, j) can be used to go the furthest away relative to where the instruction is located? Why is this true?

a) f, g, i, j → $t0, $t1, $t2, $t3

Base addr of arrays A & B → $s0, $s1

```
sll  $s2, $t2, 2     # i x 4
sll  $s3, $t3, 2     # j x 4
add  $s2, $s2, $s1   # Addr of B[i]
add  $s3, $s3, $s1   # addr of B[j]
lw   $s4, 0($s2)     # $s4 ← B[i]
lw   $s5, 0($s3)     # $s5 ← B[j]
sub  $s4, $s4, $s5   # $s4 ← B[i] - B[j]
sll  $s4, $s4, 2     # 4 x (B[i]-B[j])
add  $s4, $s4, $s0   # Addr of A[B[i]-B[j]]
lw   $s5, 0($s4)     # $s5 ← A[B[i]-B[j]]
add  $t0, $s2, $t1   # f = g + A[B[i]-B[j]]
```

b) jr $s3 (Target given in 32-bits)